PHASE

9

Synchronization and Concurrency in the Kernel

Now that tasks can be interrupted anywhere, make shared data safe: interrupt control, spinlocks, wait queues, mutexes and semaphores

Depends on Phase 8: preemptive kernel threads, `schedule()`, sleep and wake-up

Delivers Interrupt-safe critical sections, spinlocks built on the atomic `xchg` instruction, wait queues with lost-wakeup-free sleeping, mutexes, counting semaphores and condition variables, a keyboard driver that blocks instead of polling, the classic concurrency problems as kernel tests, a written lock-ordering rule checked at run time, and reference counts that cannot wrap.

1 Scope and motivation

Phase 8 introduced a new kind of bug. A task can now be interrupted between any two instructions, and another task or an interrupt handler can run and touch the same data. Before preemption, the kernel's lists and counters were only changed by one flow of control at a time. Now the run queue, the heap's free list, the PMM bitmap, the console cursor and the keyboard buffer can all be modified halfway through an update, and the result is corruption that appears rarely, at random, and never under the debugger.

This phase provides the primitives that prevent it and the rules for using them everywhere. It follows OSTEP's second big idea, concurrency, closely: build locks from a hardware atomic instruction, then build sleeping primitives on top of the locks, then use them to solve the classic problems. It also addresses deadlock, because once code takes more than one lock, the order it takes them in starts to matter.

2 Background

2.1 Race conditions and critical sections

A **race condition** is a bug whose outcome depends on the timing of concurrent events. OSTEP chapter 26 uses the simplest possible example: two threads running `counter = counter + 1`. The statement compiles to three instructions (load, add, store), and a switch between the load and the store loses an update (Table 1). The code that touches shared data is a **critical section**, and the goal is **mutual exclusion**: at most one flow of control inside it at a time.

Step	Task A	Task B	A's EAX	B's EAX	counter
1	`mov eax, [counter]`		50	–	50
2	`add eax, 1`		51	–	50
3	*timer interrupt: switch to B*		51	–	50
4		`mov eax, [counter]`	51	50	50
5		`add eax, 1`	51	51	50
6		`mov [counter], eax`	51	51	51
7	*switch back to A*		51	51	51
8	`mov [counter], eax`		51	51	**51**

Table 1: Two increments, one lost. Each task has its own saved EAX, so the switch at step 3 is invisible to A.

2.2 Where concurrency comes from in Kagan

On a single CPU there are exactly two sources of interleaving: **interrupts**, which can run a handler between any two instructions of a task, and **preemption**, which the timer interrupt causes. So on one CPU, **disabling interrupts** around a critical section is a complete lock (OSTEP §28.5). It is cheap and simple, but it doesn't scale: with several CPUs, the other CPUs keep running. That is why Kagan wraps interrupt control in a real **spinlock** built on an atomic instruction from the start. On one CPU the spinning never happens, but the code is already correct for the SMP track in Phase 15.

2.3 Atomic instructions and spinlocks

A lock needs an operation that reads and writes memory in one indivisible step. On x86, `xchg` with a memory operand is always atomic: it swaps a register with memory while locking out other CPUs. That makes it a **test-and-set**: write 1 and get the old value back; if the old value was 0, the caller has the lock (OSTEP §28.7, OSC §6.4). GCC exposes it as `__atomic_exchange_n`. Other common atomics are compare-and-swap (`lock cmpxchg`) and fetch-and-add (`lock xadd`).

2.4 Sleeping instead of spinning

Spinning wastes the CPU, and on one CPU it can never succeed while the holder isn't running. When a task has to wait for something that may take a while (a key press, a disk block, a free slot in a buffer), it should **block**: go on a **wait queue**, let the scheduler run someone else, and be put back on the ready queue by whoever makes the condition true. Two rules make this correct. First, going to sleep must be atomic with respect to the wake-up, or the result is the **lost wake-up** problem (MOS §2.3.4): the task checks the condition, the wake-up happens, and then the task sleeps forever. Second, a woken task must re-check its condition in a `while` loop, because by the time it runs, someone else may have taken what it was waiting for (OSTEP chapter 30 calls this Mesa semantics).

Three primitives, three jobs

Spinlock: short critical sections, and the only lock allowed in interrupt handlers. The holder must never sleep. **Mutex**: a sleeping lock with an owner, for longer critical sections in task context, such as the file system in Phase 13. Never from an interrupt handler. **Semaphore**: a counter with a wait queue (OSTEP chapter 31). `down` waits until the count is positive and decrements it; `up` increments it and wakes one waiter. It serves for counting resources and for signalling from an interrupt handler (which may call `up` but never `down`). **Condition variables** (`wait`, `signal`, `broadcast`) with a mutex round out the set.

2.5 Deadlock

A **deadlock** is a set of tasks each waiting for something another one holds. OSTEP chapter 32, MOS chapter 6 and OSC chapter 8 list the four conditions that must all hold for it: mutual exclusion, hold-and-wait, no preemption, and circular wait. Breaking any one prevents deadlock, and in kernels the practical choice is to break circular wait with a global **lock order**: if every piece of code takes locks in the same order, no cycle can form. MOS §6.3 also names the "ostrich algorithm", ignoring the problem, and explains when that is a reasonable choice (for the kernel, it isn't).

3 References

Weight	Source	Section	Pages	Relevance
PRIMARY	OSTEP	Ch. 26 *Concurrency: An Introduction*	273–288	Races, critical sections, atomicity.

Weight	Source	Section	Pages	Relevance
PRIMARY	OSTEP	Ch. 28 *Locks* (interrupts, test-and-set, compare-and-swap, spinning versus sleeping)	301–324	The design behind Tasks 9.1–9.2.
PRIMARY	OSTEP	Ch. 30 *Condition Variables*; Ch. 31 *Semaphores*	339–372	Producer/consumer, the `while` rule, semaphores as locks and as signals.
PRIMARY	OSTEP	Ch. 32 *Common Concurrency Problems*	373–388	Atomicity and order violations; deadlock and how to prevent it.
SECONDARY	OSC	Ch. 6 *Synchronization Tools* (critical sections, Peterson, hardware support, mutexes, semaphores, monitors, liveness)	257–288	A second, more formal treatment.
SECONDARY	OSC	§7.1–7.2 *Classic Problems*; *Synchronization within the Kernel*	289–299	What Linux and Windows use inside their kernels.
SECONDARY	MOS	§2.3 *Interprocess Communication*; §2.5 *Classical IPC Problems*	119–148, 167–172	Sleep and wakeup, semaphores, mutexes, monitors, message passing; philosophers.
SECONDARY	MOS	§6.1–6.6 *Deadlocks*	436–458	Resources, conditions, detection, avoidance, prevention.
SECONDARY	OSDI	§2.2.8 *Message Passing*; §2.6.9 *Interprocess Communication in MINIX 3*	85–88, 178–182	The alternative MINIX chose instead of kernel locks.
OPTIONAL	OSTEP	Ch. 27 *Thread API*; Ch. 29 *Lock-based Concurrent Data Structures*	289–300, 325–338	Locking real data structures well.
OPTIONAL	OSDI	§3.3 *Deadlocks*	237–252	The same theory with the I/O perspective.
PRIMARY	LKD	Ch. 9–10, *An Introduction to Kernel Synchronization*, *Kernel Synchronization Methods*	161–206	Atomics, spinlocks, semaphores, mutexes and barriers in one place.
PRIMARY	LKP	Ch. 12, *Mutex or spinlock? Which to use when* and *Locking and interrupts*	605–608, 639–647	The rule for choosing a lock, and locks shared with handlers.
PRIMARY	LKP	Ch. 13, *Using the atomic_t and refcount_t interfaces* and *Lock debugging within the kernel*	649–657, 684–703	Saturating reference counts and run-time lock checking (Tasks 9.8, 9.9).

Prior art: MINIX 3

The MINIX 3 kernel needs almost no locks: it is small, it is not preemptible, and it runs with interrupts disabled. Everything else (drivers, the file system, the process manager) runs as separate processes that cooperate through **synchronous message passing**: `send` blocks until the receiver calls `receive`, so every message is a rendezvous with no buffering (OSDI §2.2.8 and §2.6.9). `mini_send` and `mini_receive` in `kernel/proc.c (07400)` implement it, including a check that refuses a `send` which would close a cycle of processes sending to each other: deadlock detection built into the IPC primitive itself.

Linux, for comparison. LKP Linux checks its lock order at run time: **lockdep** records which locks are held when another is taken and reports the first order violation, long before it deadlocks. It

also replaced plain atomic counters for object lifetimes with `refcount_t`, which saturates instead of wrapping to zero. Tasks 9.8 and 9.9 bring both ideas to Kagan.

4 Implementation

Task 9.1 Interrupt control that nests

Replace every bare `cli/sti` with a pair that saves and restores the previous state. A function that disables interrupts must not enable them on exit if they were already disabled when it was called.

include/arch/irq.h

```
static inline uint32_t irq_save(void)
{
    uint32_t flags;
    __asm__ volatile ("pushf; pop %0; cli" : "=r"(flags) : : "memory");
    return flags;
}

static inline void irq_restore(uint32_t flags)
{
    if (flags & 0x200)                      /* IF was set: enable interrupts again */
        __asm__ volatile ("sti" : : : "memory");
}
```

Task 9.2 Spinlocks

A spinlock disables interrupts (on one CPU, that is what actually excludes everyone else) and then takes an atomic flag, so the same code stays correct on several CPUs. Record the owner: taking a lock you already hold is a guaranteed deadlock, and an assertion turns it into an immediate, readable panic. The `"memory"` clobbers and the acquire/release orderings stop the compiler from moving accesses out of the critical section.

kernel/sync/spinlock.c

```
typedef struct {
    volatile uint32_t locked;
    uint32_t          saved_flags;          /* interrupt state of the holder before locking */
    struct task      *owner;                /* debugging: who holds it                      */
    const char       *name;
} spinlock_t;

void spin_lock(spinlock_t *l)
{
    uint32_t flags = irq_save();
    ASSERT(!(l->locked && l->owner == current));          /* already mine: deadlock */
    while (__atomic_exchange_n(&l->locked, 1, __ATOMIC_ACQUIRE))
        __asm__ volatile ("pause");                      /* only ever spins on SMP */
    l->saved_flags = flags;
    l->owner = current;
}

void spin_unlock(spinlock_t *l)
{
    ASSERT(l->locked && l->owner == current);
    uint32_t flags = l->saved_flags;
    l->owner = NULL;
    __atomic_store_n(&l->locked, 0, __ATOMIC_RELEASE);
    irq_restore(flags);
}
```

```
#define ASSERT_HELD(l) ASSERT((l)->locked && (l)->owner == current)
```

The owner field buys you one more check. Many functions are only correct when their caller already holds a lock: `make_ready` touches the run queue, a helper that unlinks a buffer touches the cache's lists. A comment saying "caller holds `sched_lock`" is easy to ignore, so make the first line of such a function `ASSERT_HELD(&sched_lock)`. A caller that forgot the lock now panics on the first call, instead of corrupting a list once a week. Mutexes get the same macro over their owner field.

Verify: Taking the same lock twice panics with the lock's name. Releasing locks in the reverse order they were taken restores the original interrupt state at every level.

Task 9.3 Wait queues without lost wake-ups

A wait queue is a list of blocked tasks. `sleep_on(wq, lock)` is called with a spinlock held, which on one CPU means interrupts are off. It marks the task blocked and queues it *before* releasing the lock, so a wake-up can't slip in between. Then it schedules, and it takes the lock again before returning, so the caller can re-check its condition safely. `wake_up_one` and `wake_up_all` move tasks from the wait queue to the ready queue; they never block, so interrupt handlers may call them.

kernel/sync/waitq.c

```
struct wait_queue { struct list_node tasks; };

/* Caller holds lk. Returns with lk held again, after a wake_up. */
void sleep_on(struct wait_queue *wq, spinlock_t *lk)
{
    uint32_t caller_flags = lk->saved_flags;
    current->state = TASK_BLOCKED;
    list_add_tail(&wq->tasks, &current->node);
    lk->owner = NULL;
    __atomic_store_n(&lk->locked, 0, __ATOMIC_RELEASE);  /* release, keep interrupts off */
    schedule();                                           /* returns after we are woken  */
    spin_lock(lk);
    lk->saved_flags = caller_flags;                       /* keep the caller's IF state  */
}

void wake_up_one(struct wait_queue *wq)
{
    if (!list_empty(&wq->tasks)) {
        struct task *t = container_of(wq->tasks.next, struct task, node);
        list_del(&t->node);
        make_ready(t);
    }
}
```

Task 9.4 Semaphores, mutexes and condition variables

Build the three sleeping primitives on spinlocks and wait queues. The semaphore is shown below. A mutex is a semaphore with an initial count of one plus an owner field and an assertion that only the owner unlocks. A condition variable is a wait queue used together with a mutex: `cv_wait` releases the mutex and sleeps atomically, and re-acquires the mutex before returning.

kernel/sync/semaphore.c

```
struct semaphore {
    int               count;
    spinlock_t        lock;
    struct wait_queue waiters;
};
```

```
void sem_down(struct semaphore *s)          /* task context only: may sleep */
{
    spin_lock(&s->lock);
    while (s->count == 0)                   /* while, not if: re-check after waking */
        sleep_on(&s->waiters, &s->lock);
    s->count--;
    spin_unlock(&s->lock);
}

void sem_up(struct semaphore *s)            /* safe from interrupt handlers */
{
    spin_lock(&s->lock);
    s->count++;
    wake_up_one(&s->waiters);
    spin_unlock(&s->lock);
}
```

Task 9.5 A keyboard that blocks

Rewrite the keyboard buffer as a producer/consumer pair. The interrupt handler is the producer: it puts the character in the ring buffer and calls `sem_up` on a semaphore that counts available characters. `kbd_getchar` is the consumer: `sem_down`, then take a character. The `sti; hlt` loop from Phase 4 disappears; a task waiting for a key now simply sleeps and costs nothing.

Verify: With the monitor waiting for input and no other task ready, `ps` shows the monitor as blocked and the idle task running.

Task 9.6 Make the classic problems into tests

Add these to `make test`. Each one should run long enough (a few seconds) to expose timing bugs.

Test	What it shows
`race_counter`	four tasks increment a shared counter 100 000 times each. Without a lock the total comes out short; with a spinlock it is exactly 400 000. To make the race visible on one CPU, widen the window between load and store with a short busy loop.
`bounded_buffer`	three producers and three consumers share a 16-slot buffer, using semaphores for empty slots, full slots and mutual exclusion (OSTEP chapter 31). Every item produced is consumed exactly once.
`philosophers`	five tasks, five forks. Taking forks in a fixed global order (lowest number first) runs without deadlock; a version without ordering, run once, freezes, which shows the test can detect deadlock.
`cv_ping_pong`	two tasks alternate strictly using a mutex and a condition variable, 10 000 rounds.

The helpers these tests use to touch shared state, such as the bounded buffer's put and get, start with `ASSERT_HELD`, like the kernel functions they imitate.

Verify: All four tests pass repeatedly. The unlocked version of `race_counter` fails, which proves the test can see races. Remove the `spin_lock` before one call to the bounded buffer's put helper, and its `ASSERT_HELD` fires on the first call.

Task 9.7 Audit the kernel and write the lock order

Go through every shared structure in the kernel and give it an owner. Write the result into a comment block in `include/locks.h` and follow it from now on. Locks are always taken from top to bottom in the table, never upward.

Data	Protected by	Notes
task list, run queue, sleep queue	`sched_lock` (spinlock)	taken inside `schedule()`, `make_ready()` and the timer handler
keyboard buffer	its semaphore and a spinlock	producer is the IRQ handler
console and `kprintf`	`console_lock` (spinlock)	also used from interrupt context, so it must be a spinlock
kernel heap and slab caches	`heap_lock` (spinlock)	leaf lock: never call anything that takes another lock while holding it
PMM free lists	`pmm_lock` (spinlock)	leaf lock

Task 9.8 Reference counts that cannot wrap

Phase 11 counts how many address spaces share a frame, and Phase 13 how many descriptors share an open file. A plain counter is dangerous there: one missing increment or one extra decrement sends it to zero while the object is still in use, and the kernel frees memory that someone still writes to. Linux moved such counters from `atomic_t` to `refcount_t` (LKP ch. 13), which refuses the impossible cases: incrementing a count that is already zero (the object is being freed) and overflowing past the maximum both trigger a warning and leave the count stuck at a saturation value, so the object leaks instead of being freed early. A leak is a bug; a use-after-free is a security hole.

include/refcount.h

```
#define REFCOUNT_SATURATED 0xC0000000u

typedef struct { volatile uint32_t refs; } refcount_t;

static inline void refcount_inc(refcount_t *r)
{
    unsigned long f = irq_save();               /* one CPU: masking interrupts is enough */
    if (r->refs == 0 || r->refs >= REFCOUNT_SATURATED) {
        pr_warn("refcount: inc on %lu at %p\n", r->refs, r);
        r->refs = REFCOUNT_SATURATED;           /* never free this object */
    } else {
        r->refs++;
    }
    irq_restore(f);
}

static inline bool refcount_dec_and_test(refcount_t *r)   /* true: the caller frees it */
{
    unsigned long f = irq_save();
    bool last = false;
    if (r->refs == 0)
        pr_warn("refcount: dec below zero at %p\n", r);
    else if (r->refs != REFCOUNT_SATURATED)
        last = (--r->refs == 0);
    irq_restore(f);
    return last;
}
```

Verify: A test that increments from zero, and one that decrements an object twice, each print a warning and leave the count saturated; the object is never freed. The normal pattern (create at 1, share, release everywhere) frees it exactly once.

Task 9.9 Check the lock order at run time

The table above is only a promise until something checks it. Linux's **lockdep** (LKP ch. 13) watches every acquisition and reports the first order violation, long before it turns into a deadlock that

happens once a week. A much smaller version suffices for Kagan: give each lock a **rank** from the table, keep a small stack of the locks the CPU holds, and in debug builds panic when a lock is taken whose rank is not greater than the rank of the lock on top. Unlike lockdep, which learns the order from what it sees, the Kagan checker enforces the documented order.

kernel/sync/lockcheck.c (with LOCK_DEBUG)

```
static struct spinlock *held[16];
static int nheld;

void lockcheck_acquire(struct spinlock *l)      /* called by spin_lock and mutex_lock */
{
    if (nheld > 0 && l->rank <= held[nheld - 1]->rank)
        panic("lock order: %s (rank %d) taken while holding %s (rank %d)",
              l->name, l->rank, held[nheld - 1]->name, held[nheld - 1]->rank);
    ASSERT(nheld < 16);
    held[nheld++] = l;
}

void lockcheck_release(struct spinlock *l)
{
    ASSERT(nheld > 0 && held[nheld - 1] == l);  /* release in reverse order */
    nheld--;
}
```

Mutexes need the same check, but their holder can sleep: keep the held-lock stack in `struct task` instead of a global array once mutexes take part. Run the suite this way in CI as well: add a step `make clean && make test LOCK_DEBUG=1` to the Phase 7 workflow (the Makefile turns the variable into `-DLOCK_DEBUG`), so the classic problems and every other test run under the lock-order checker on every push.

Verify: A test that takes `heap_lock` and then `sched_lock` panics with both names in debug builds; the whole `make test` suite runs with `LOCK_DEBUG` enabled, in CI as well as locally, and reports no violation.

5 Testing and debugging

Concurrency bugs hide from debuggers because stopping the machine changes the timing. Rely on assertions instead: check lock ownership, check that interrupts are disabled where you expect them to be (`ASSERT(!(read_eflags() & 0x200))`), and run tests many times. Changing the timer frequency (10 Hz, 1000 Hz) shuffles the interleavings and often exposes a race that was hiding.

Symptom	Likely cause	Resolution
System freezes the first time an IRQ handler runs	The handler took a mutex or called `sem_down`, which sleeps in interrupt context	Handlers may only take spinlocks and call `wake_up`/`sem_up`. Assert that you are not in an interrupt in every sleeping function.
Deadlock when `kprintf` is called from an interrupt	The interrupted task held `console_lock`, and the lock didn't disable interrupts	Spinlocks must disable interrupts: that is exactly what `spin_lock` does.
A task sleeps forever although the event happened	Lost wake-up: the condition was checked, then the wake-up came, then the task slept	Queue the task before releasing the lock, as `sleep_on` does.
Consumers take items that aren't there	`if` instead of `while` around the wait	Always re-check the condition in a `while` loop.
Interrupts stay off after some code path	An early `return` skipped `spin_unlock` or `irq_restore`	One exit path per locked function; assert IF at the top of the idle loop.

Symptom	Likely cause	Resolution
Occasional hang under load	Two locks taken in opposite orders	Follow the lock order table; see Task 9.9 and Extensions for automatic checking.

6 Exit criteria

Exit criteria: Safe concurrency

- ☐ `irq_save`/`irq_restore`, spinlocks, wait queues, semaphores, mutexes and condition variables are implemented.
- ☐ The unlocked counter test loses updates; the locked one is exact.
- ☐ Bounded buffer, philosophers and ping-pong tests pass repeatedly in `make test`.
- ☐ The keyboard driver blocks the reader on a semaphore; no busy waiting remains in the kernel.
- ☐ Every shared structure has a documented lock, the lock order is written down, and `LOCK_DEBUG` builds enforce it.
- ☐ Functions that need a lock assert it with `ASSERT_HELD`, and CI runs the classic-problem tests with the lock-order checker.
- ☐ `refcount_t` warns and saturates instead of wrapping, and the reference-count tests pass.

Review questions

1. Why is disabling interrupts a complete lock on one CPU but not on several?
2. Write test-and-set with `xchg`. Why must the exchange be atomic, and what does “atomic” mean on a multiprocessor?
3. What is the lost wake-up problem, and which line of `sleep_on` prevents it?
4. Compare a spinlock, a mutex, a semaphore and a condition variable. Which may be used in an interrupt handler?
5. Why must a woken task re-check its condition in a `while` loop?
6. Name the four conditions for deadlock. Which one does a lock order break, and how?
7. MINIX uses synchronous message passing instead of shared-memory locks. What does it gain and what does it lose?

7 Extensions

Build a small *lock validator*: give each lock a class number, record the order in which each task acquires classes, and panic the first time an order inversion appears, even if it didn’t deadlock that time. Linux’s lockdep works this way. Implement reader-writer locks (OSTEP chapter 31) for data that is read far more often than written. Or add MINIX-style synchronous messages (`send`, `receive`, `sendrec`) as a second IPC mechanism between kernel threads, and compare the code each style produces.

Next: Phase 10 User Mode and System Calls →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and OSTEP are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.